

Нейронні SDF для початківців: від основ до власної реалізації

Детальний посібник з нейронного представлення 3D-геометрії на основі Instant-NGP, з поясненням кожного кроку математики й кожної модифікації, які я зробив у своїй реалізації.

Автор: Ігор Іванишин Дата: 18 травня 2026

Зміст

1. Що таке 3D-геометрія і як її зберігати
 2. Що таке SDF (знакова функція відстані)
 3. Чому SDF корисна і де вона застосовується
 4. Нейронні мережі: мінімальний словник
 5. Огляд методів навчання SDF
 6. Instant-NGP крок за кроком (наша основа)
 7. Мої модифікації
 8. Як тренується модель
 9. Як швидко обчислюється inference
 10. Підсумок результатів
-

Частина 1. Як комп'ютер бачить 3D

1.1. Що таке 3D-об'єкт у файлі

Уявіть, що ви тримаєте в руках статую. У комп'ютері її не можна "тримати" — потрібно якось закодувати її форму у вигляді чисел. Найпоширеніший спосіб — **полігональна сітка** (англ. *mesh*).

Mesh складається з трьох речей:

1. **Вершини** (vertices) — точки у тривимірному просторі. Кожна вершина має три координати (x, y, z) . Наприклад, вершина кута столу може мати координати $(0.5, 0.3, 1.0)$.

2. **Ребра** (edges) — відрізки між парами вершин. Зазвичай у файлі їх явно не зберігають, бо вони відновлюються з трикутників.
3. **Грані** (faces) — найчастіше **трикутники**, кожен з яких задається трьома індексами вершин. Якщо вершини під номерами 5, 12, 78 утворюють трикутник, то face — це просто запис (5, 12, 78).

Файл `.obj` (формат, який нам дали в задачі) виглядає приблизно так:

```
v 0.5 0.3 1.0      # вершина №1, координати (0.5, 0.3, 1.0)
v 0.5 0.3 0.0      # вершина №2
v 0.5 0.7 0.5      # вершина №3
...
f 1 2 3            # трикутник з вершин 1, 2, 3
f 2 3 4            # трикутник з вершин 2, 3, 4
```

Усе. Сотні тисяч таких рядків — і ви отримаєте опис будь-якої складної форми. Mesh кота, мотоцикла, людської голови, статуї — все це лише списки вершин і трикутників.

1.2. Чим погано просто зберігати mesh

Mesh — це **дискретне** представлення. У нас є точно визначений набір трикутників. Будь-яке питання типу "Чи точка $(0.234, 0.871, 0.512)$ всередині цього об'єкта?" вимагає алгоритмів, які перевіряють перетин променя з трикутниками. Це працює, але:

- **Розмір.** Один mesh може важити мегабайти або десятки мегабайт.
- **Дискретність.** Між трикутниками є шви; між сусідніми трикутниками може бути "тріщина".
- **Складність операцій.** Перетин двох mesh, згладжування, морфінг, ray-marching рендеринг — усе це вимагає різних спеціалізованих алгоритмів.
- **Не диференційовний.** Не можна "трохи зрушити" один трикутник у напрямку градієнта якоїсь цільової функції — mesh не має поняття градієнта.

Альтернативні представлення:

- **Воксельна сітка** (3D-аналог пікселів) — простіше, але неефективно щодо пам'яті при високій роздільності.
 - **Point cloud** — просто хмара точок без зв'язків. Втрачає інформацію про поверхню.
 - **Параметричні поверхні** (NURBS) — гнучкі, але вимагають експертів-художників.
 - **Неявні функції** (implicit) — те, що ми зараз вивчатимемо. Це і є SDF.
-

Частина 2. Знакова функція відстані (SDF)

2.1. Інтуїтивне визначення

SDF (Signed Distance Function — "знакова функція відстані") — це **функція**, яка для будь-якої точки $\mathbf{x} = (x, y, z)$ у просторі повертає одне число:

$$f(\mathbf{x}) = \text{знакова відстань від } \mathbf{x} \text{ до поверхні об'єкта}$$

Конкретно:

- Якщо точка **поза** об'єктом, $f(\mathbf{x}) > 0$, і число дорівнює звичайній (евклідовій) відстані до найближчої точки поверхні.
- Якщо точка **на** поверхні, $f(\mathbf{x}) = 0$.
- Якщо точка **всередині** об'єкта, $f(\mathbf{x}) < 0$, і число — це від'ємна відстань до найближчої точки поверхні.

Це і є "знакова" — той самий модуль відстані, але зі знаком, який каже, всередині ви чи зовні.

2.2. Приклад: SDF одиничної сфери

Для сфери з центром у початку координат та радіусом 1:

$$f(\mathbf{x}) = |\mathbf{x}| - 1 = \sqrt{x^2 + y^2 + z^2} - 1$$

Перевіримо:

- Точка $(2, 0, 0)$: $f = \sqrt{4} - 1 = 1$. Тобто відстань 1, точка зовні. ✓
- Точка $(0, 0, 0)$: $f = 0 - 1 = -1$. Центр сфери, відстань всередині = 1. ✓
- Точка $(1, 0, 0)$: $f = 1 - 1 = 0$. Точка на поверхні. ✓

Для сфери це красива замкнена формула. Для складніших форм (типу тих 50 mesh-ів, які нам дали) аналітичної формули не існує — потрібно або обчислювати чисельно (повільно), або **апроксимувати нейронною мережею**.

2.3. Чому ця функція "неявно" задає об'єкт

Сама поверхня об'єкта — це **множина нульового рівня** функції:

$$\text{поверхня} = \{\mathbf{x} \in \mathbb{R}^3 : f(\mathbf{x}) = 0\}$$

Тобто ми не зберігаємо самі трикутники — ми зберігаємо функцію, нульовий рівень якої і є поверхнею. Це називається **неявним представленням** (implicit representation).

Якщо ми хочемо побачити меш — застосовуємо алгоритм **Marching Cubes**, який бере регулярну сітку точок, обчислює f у кожній з них, і там, де знак змінюється з $+$ на $-$, будує трикутники. Це переходить з неявного представлення назад у явний mesh.

2.4. Цікаві властивості справжньої SDF

Справжня SDF (не апроксимація) задовольняє **ейконал-рівняння**:

$$\|\nabla f(\mathbf{x})\| = 1 \quad \text{\textit{майже всюди}}$$

Тобто градієнт (вектор частинних похідних) має одиничну довжину. Чому? Бо SDF — це **відстань**, і якщо ви рухаєтесь на 1 метр у будь-якому напрямку, відстань до поверхні може змінитися щонайбільше на 1 метр (а у найкращому напрямку — рівно на 1).

Цю властивість можна використовувати як регуляризатор під час навчання (нав'язуємо моделі рости в потрібному темпі). У нашому проєкті ми це пробували і виявилось, що для тонких mesh-ів воно шкодить — про це пізніше.

Частина 3. Навіщо нам SDF

Якщо у нас уже є mesh, чому б ми не використовували просто mesh?

3.1. Render через ray marching

Класичний рендеринг (растеризація) проєктує трикутники на екран. Це швидко, але вимагає mesh-у.

Ray marching — інший спосіб: для кожного пікселя екрана пускаємо промінь і "крокуємо" вздовж нього, на кожному кроці питаючи SDF, наскільки ми далеко від поверхні. SDF гарантує: якщо $f(\mathbf{x}) = d$, то на d кроків уперед поверхні нема. Тому можна стрибати:

```
1. position = camera
2. while not hit:
3.     d = sdf(position)
4.     if d < 0.001: hit!           # дотик до поверхні
5.     if d > 100: skip             # пішли в нескінченність
6.     position += direction * d    # стрибок на безпечну відстань
```

Це працює лише з SDF, не з mesh. І це базис для багатьох красивих ефектів (м'які тіні, ambient occlusion, soft refraction).

3.2. Детекція колізій

В іграх або симуляціях — питання "чи цей об'єкт зіткнувся з іншим" зводиться до:

$$\text{\textit{колізія}} \iff \exists \mathbf{x} : f_A(\mathbf{x}) < 0 \wedge f_B(\mathbf{x}) < 0$$

тобто є точка, яка одночасно всередині обох. З mesh — це складна геометрична задача. З SDF — простий запит.

3.3. Boolean операції

Хочете об'єднання двох об'єктів? $f_{\text{union}} = \min(f_A, f_B)$. Перетин?

$f_{\text{intersection}} = \max(f_A, f_B)$. Віднімання? $f_{\text{diff}} = \max(f_A, -f_B)$.

З mesh-ами це окремий розділ обчислювальної геометрії з купою тонкощів. З SDF — одна формула.

3.4. Компактність

Mesh на сотні тисяч трикутників = мегабайти. Нейронна SDF з 130 000 параметрів = ~256 КБ і покриває всю геометрію. Це наш бюджет 1 МБ — і нейронна SDF з'являється саме тому, що вона **дуже компактна**.

3.5. Диференційовність

Нейронна функція f_{θ} диференційовна (вона ж нейромережа). Це означає, що можна обчислити, як змінити параметри θ , щоб поверхня змістилася в потрібному напрямку. Це відкриває двері до:

- Реконструкції форм з знімків (inverse rendering)
- Оптимізації форм під фізичні обмеження
- Прогресивного моделювання

Частина 4. Нейронні мережі — на двох сторінках

Якщо ви взагалі не знаєте, що таке нейронні мережі — ось мінімум.

4.1. Що це таке

Нейронна мережа — це параметризована функція. Вона приймає на вхід числа $\mathbf{x}$, виконує над ними низку послідовних математичних операцій, і повертає інші числа на виході. Параметри (зазвичай позначаються θ або W і b) — це числа, які ми **навчаємо** так, щоб функція виконувала корисну задачу.

Найпростіший приклад — **лінійний шар** (Linear, або Dense, або Fully-Connected layer):

$$\mathbf{y} = W \mathbf{x} + \mathbf{b}$$

де W — матриця ваг, $\mathbf{b}$ — вектор зсувів. Множення матриці на вектор — це звичайна шкільна формула, але якщо W великий (скажімо, 64×64), сама матриця має 4 096 параметрів.

Послідовно з'єднуючи кілька таких шарів, перемешовуючи їх **нелінійностями** (наприклад, ReLU — про що далі), отримуємо потужну апроксимаційну машину, яка теоретично може представити майже будь-яку функцію.

4.2. Що таке ReLU

ReLU (Rectified Linear Unit) — найпростіша і найпопулярніша нелінійність:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{якщо } x > 0 \\ 0 & \text{якщо } x \leq 0 \end{cases}$$

Тобто: лишити число як є, якщо воно додатне; зробити нулем, якщо від'ємне. Просто. Чому це працює? Бо без такої нелінійності будь-який стек лінійних шарів все одно еквівалентний одному лінійному шару (множення матриць). Нелінійність "розриває" цю еквівалентність і дає мережі змогу представляти складніші функції — зокрема, нелінійні (наприклад, такі, що мають "вигини" або змінюють поведінку в різних регіонах простору).

4.3. Що таке "навчити" мережу

Маємо набір прикладів (x_i, y_i) — вхід і очікуваний вихід. Мета — знайти такі параметри θ , щоб $f_\theta(x_i) \approx y_i$ для всіх i .

Для цього визначаємо **функцію втрат** (loss function), яка вимірює "наскільки погано":

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_i \ell(f_\theta(x_i), y_i)$$

Найпопулярніші ℓ :

- **L1 (Mean Absolute Error):** $\ell = |f_\theta(x) - y|$ — модуль різниці.
- **L2 (Mean Squared Error):** $\ell = (f_\theta(x) - y)^2$ — квадрат різниці.

У нашому SDF-проекті ми використовуємо L1, бо вона менш чутлива до викидів (одна шалена пророчена точка не псує градієнт усього батчу).

4.4. Як знаходять параметри: градієнтний спуск

Маючи $\mathcal{L}(\theta)$, ми обчислюємо її **градієнт** $\nabla_\theta \mathcal{L}$ — вектор частинних похідних по кожному параметру. Потім робимо крок проти градієнта:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta \mathcal{L}$$

де η (η) — **швидкість навчання** (learning rate), невелике додатне число (типово 10^{-3} до 10^{-2}).

Чому проти градієнта? Градієнт показує напрям найшвидшого зростання функції. Якщо ми хочемо зменшити loss, рухаємось у *протилежному* напрямі.

Adam — це конкретний оптимізатор, який вдосконалює простий градієнтний спуск:

- запам'ятовує "момент" (середнє по останніх кроках), щоб уникати дрижання,
- адаптивно зменшує крок для параметрів, у яких градієнт часто змінюється,
- адаптивно збільшує крок для параметрів, у яких градієнт стабільний.

Adam — стандартний вибір для більшості нейромережних задач, ним користуємось і ми.

4.5. Що таке перенавчання (overfitting)

Якщо параметрів багато, а прикладів мало, модель може **завчити** точні відповіді для тренувальних прикладів, але втратити узагальнення. Перевірка — окремий **тестовий набір**, який модель ніколи не бачила. Якщо точність на тренуванні висока, а на тесті низька — це overfitting.

В нашому проєкті це стало центральною проблемою, тому ми перейшли від кешованих даних до **online resampling** — кожен крок ми генеруємо свіжі точки, тож модель не може їх "запам'ятати".

Частина 5. Огляд методів

Перш ніж занурюватись у Instant-NGP, коротко про сімейство методів навчання SDF. Хто був до нас:

5.1. DeepSDF (2019)

Великий MLP (8 шарів $\times$ 512 нейронів), який вчиться відображати координату $\mathbf{x}$ у число SDF. Класика. Проблеми для нас:

- ~ 1.8 мільйона параметрів = 7 МБ $\rightarrow$ за межі бюджету.
- ~ 5 мікросекунд на запит $\rightarrow$ за межі бюджету.

5.2. SIREN (2020)

Той самий MLP, але з **синусовими активаціями** замість ReLU. Працює дуже добре для високочастотних деталей. Все одно це MLP, все одно $\sim 250\text{K}$ параметрів = 1 МБ — на межі.

5.3. NGLOD (2021)

Перший справді гібридний підхід. Геометрія зберігається в **октодереві** з feature-векторами на вузлах; маленький MLP читає ці фічі та декодує SDF. Дуже ефективно за пам'яттю, але октодерево складно реалізувати ефективно (вказівники на батьків/дітей, обходи дерева — все це повільно на сучасних процесорах).

5.4. Instant-NGP (2022)

Те саме, що NGLOD, але октодерево замінене на **багаторівневі хеш-таблиці**. Запит за константний час, без обходу дерева, без вказівників. Найшвидший і найкомпактніший метод серед опублікованих. Це **наш основний reference**.

Частина 6. Instant-NGP — як він працює

Цей розділ — детальний крок-за-кроком розбір методу, на якому будується наша реалізація. Зрозуміти його — означає зрозуміти 80% коду.

6.1. Загальна ідея

Уявіть, що ми хочемо зберегти неперервне поле (SDF) над тривимірним кубом $[-1, 1]^3$. Найпростіший спосіб — **сітка**: ділимо куб на $N \times N \times N$ маленьких комірок, у кожній зберігаємо число (значення SDF у центрі цієї комірки). Для $N = 256$ це $16\text{ мільйонів чисел} = 64\text{ МБ}$. Забагато для нашого бюджету.

Хитрість Instant-NGP:

1. Замість одного "правильного" розміру N — використовуємо **багато рівнів** $N_0 < N_1 < \dots < N_{L-1}$.
2. На кожному рівні зберігаємо не SDF-числа в комірках, а **feature-вектори** (короткі вектори чисел, типово $F = 2$).
3. Кожен рівень має **хеш-таблицю** фіксованого розміру T (типово $T = 8192$). Якщо вокселів на цьому рівні більше за T , кілька вокселів просто отримують одну й ту саму комірку в таблиці — це **колізія**.
4. Конкатенуємо feature-вектори з усіх рівнів і пропускаємо через дуже маленький MLP для декодування у скаляр SDF.

Чому це працює, попри колізії? На низьких рівнях (N малий, до $T = 8192$ комірок не доходить) колізій нема, таблиця поводить себе як щільна сітка. На високих рівнях колізії неминучі, але модель *вчиться розміщувати* інформацію в тих хеш-комірках, де вона важлива (біля поверхні), і "знущатися" з порожніх вокселів через колізії в одній клітинці.

6.2. Детальний крок-за-кроком forward pass

Розглянемо, як обчислюється SDF для **однієї точки** $\mathbf{x} \in [-1, 1]^3$.

Конфігурація (попередньо вибрана)

L	= 8	кількість рівнів
T	= $2^{13} = 8192$	розмір хеш-таблиці на рівні
F	= 2	розмір feature-вектора у кожній комірці
N _{min}	= 8	найгрубша роздільність
N _{max}	= 128	найтонша роздільність
hidden	= 16	ширина прихованого шару декодера

Резолюції рівнів обчислюються одноразово:

$$b = (N_{\max} / N_{\min})^{1/(L-1)} = (128/8)^{1/7} \approx 1.486$$

$$N_{\ell} = \lfloor N_{\min} \cdot b^{\ell} \rfloor \quad \rightarrow \quad N = [8, 11, 17, 26, 39, 58, 86, 128]$$

Чому геометрична прогресія, а не арифметична? Бо нам потрібно покрити різні масштаби: одночасно загальну форму (потрібен великий, грубий рівень) і дрібні деталі (потрібен дрібний рівень). Геометрична прогресія розподіляє роздільності по логарифмічній шкалі — ідеально для багаточастотного аналізу.

Чому 8 рівнів, а не 16 чи 4? Емпірично. У статті Instant-NGP пропонували 16. Ми зменшили до 8 заради бюджету пам'яті. Якість майже не страждає.

Крок 1. Обрізаємо точку до куба $[-1, 1]^3$

```
x = clamp(x, -1, 1)  # якщо якась координата вийшла за межі, повертаємо до межі
```

Чому? Хеш-таблиці визначені лише для точок усередині куба. Якщо запит прийшов за межами (наприклад, $(1.3, 0.1, 0.1)$), ми просто округлюємо до межі.

Крок 2. Афінне перетворення $[-1, 1]^3 \rightarrow [0, 1]^3$

$$u = (x + 1) / 2$$

Чому? Бо сіткові координати завжди йдуть від 0, не від -1. Це проста зміна системи координат: ми зсуваємо центр з $(0,0,0)$ у $(0.5, 0.5, 0.5)$ і нормуємо.

Якщо вхід був $\mathbf{x} = (0.20, -0.30, 0.55)$, то $\mathbf{u} = (0.60, 0.35, 0.775)$.

Крок 3. Для кожного рівня ℓ (8 ітерацій)

Розглянемо приклад для рівня $\ell = 5$ з $N_5 = 58$.

3.1. Масштабуємо до координат сітки

$$\text{scaled} = u * (N - 1) = (0.60, 0.35, 0.775) * 57 = (34.2, 19.95, 44.175)$$

Чому $(N - 1)$, а не N ? Бо у нас є N комірок, але **$N+1$ вершин**. Точка $u = 0$ має відповідати індексу 0; точка $u = 1$ — індексу $N - 1$ (остання вершина).

3.2. Знаходимо комірку

$$\text{base} = \text{floor}(\text{scaled}) = (34, 19, 44)$$

`base` — це індекс нижнього-лівого-заднього кута комірки, в якій лежить наша точка.

3.3. Знаходимо позицію всередині комірки

$$\text{frac} = \text{scaled} - \text{base} = (0.2, 0.95, 0.175)$$

`frac` — це три числа в $[0, 1)$, які показують, де всередині одиничного кубика лежить наша точка. Будемо використовувати їх як **вагові коефіцієнти інтерполяції**.

3.4. Шукаємо 8 кутів комірки

Кубічна комірка має 8 кутів. Їх індекси — це всі комбінації базового кута з $\{\text{base}, \text{base}+1\}$:

```
8 corners = [
    (34, 19, 44), (35, 19, 44), (34, 20, 44), (35, 20, 44),
    (34, 19, 45), (35, 19, 45), (34, 20, 45), (35, 20, 45)
]
```

3.5. Хешуємо кожен кут

Хеш-функція з статті Instant-NGP:

$$h(i, j, k) = (i \cdot p_0) \oplus (j \cdot p_1) \oplus (k \cdot p_2) \bmod T$$

де $p_0 = 1$, $p_1 = 2^{654}, 435, 761$, $p_2 = 805, 459, 861$, а $\oplus$ — побітове XOR.

Що таке XOR? Для двох бітів: $a \oplus b = 1$, якщо вони різні; 0 , якщо однакові:

```
0 ⊕ 0 = 0
0 ⊕ 1 = 1
1 ⊕ 0 = 1
1 ⊕ 1 = 0
```

Для цілих чисел XOR — це побітова операція: береш два числа в двійковій формі і робиш XOR кожної відповідної пари бітів.

Чому саме ці прості числа? Це числа Кнута для просторового хешування. Дві властивості, які вони мають:

1. Великі (порядку 2^{32}), щоб помноження "розподіляло" біти.
2. Взаємно прості, щоб XOR трьох добутоків добре змішував координати.

Що таке $h \% T$? Залишок від ділення на T . Це гарантує, що результат — індекс у межах таблиці ($[0, T-1]$).

Маленька оптимізація: $T = 2^{13}$ — степінь двійки, тому $h \% T == h \& (T - 1)$. Побітове AND з $T-1$ — це одна процесорна інструкція, тоді як модуль — кілька. У нашому inference-коді ми використовуємо AND.

3.6. Дістаємо 8 feature-векторів

Кожен хеш дає індекс у таблиці, з якої ми читаємо F -вимірний вектор:

```
f_000 = features[5][h_000] # це вектор, скажімо (0.043, -0.118)
f_100 = features[5][h_100] # (-0.021, 0.157)
... (8 таких векторів)
```

3.7. Трилінійна інтерполяція

Маємо 8 значень у 8 кутах кубика і хочемо плавно обчислити значення в довільній внутрішній точці. **Трилінійна інтерполяція** — це послідовне застосування лінійної інтерполяції тричі: вздовж X , потім вздовж Y , потім вздовж Z .

Спочатку нагадаємо, що таке **лінійна інтерполяція**: маючи два значення A і B на кінцях відрізка і параметр $t \in [0, 1]$, інтерпольоване значення:

$$\text{lerp}(A, B, t) = (1 - t) \cdot A + t \cdot B$$

При $t = 0$ повертає A ; при $t = 1$ повертає B ; при $t = 0.5$ — середнє. Лінійне змішування.

Білінійна інтерполяція — двократне застосування на квадраті (4 кути). **Трилінійна** — трикратне на кубі (8 кутів).

Розпишемо алгоритм для 8 кутів. Позначимо $f_{\{abc\}}$ значення в кутку з координатами (a, b, c) , де $a, b, c \in \{0, 1\}$. Параметри: f_x, f_y, f_z (це наш $\frac{f}{\text{frac}}$).

Прохід 1 (по X): інтерполюємо вздовж X — отримуємо 4 значення на ребрах:

$$\begin{aligned} c_{00} &= (1 - f_x) \cdot f_{000} + f_x \cdot f_{100} \\ c_{10} &= (1 - f_x) \cdot f_{010} + f_x \cdot f_{110} \\ c_{01} &= (1 - f_x) \cdot f_{001} + f_x \cdot f_{101} \\ c_{11} &= (1 - f_x) \cdot f_{011} + f_x \cdot f_{111} \end{aligned}$$

Прохід 2 (по Y): інтерполюємо ці 4 значення в пари — отримуємо 2:

$$c_0 = (1 - f_y) \cdot c_{00} + f_y \cdot c_{10} \\ c_1 = (1 - f_y) \cdot c_{01} + f_y \cdot c_{11}$$

Прохід 3 (по Z): інтерполюємо два числа в одне:

$$f_{\text{ell}} = (1 - f_z) \cdot c_0 + f_z \cdot c_1$$

Результат — F -вимірний вектор. Це **feature-вектор від рівня ℓ** для нашої точки.

Чому саме трилінійна, а не якась інша? Бо:

1. Вона **неперервна** (точки на межах сусідніх вокселів отримують однакові значення).
2. Вона **дешева** — кілька множень і додавань.
3. Її **похідна** обчислюється легко (потрібно для backprop).
4. Вона **локально лінійна** — це означає, що поведінка функції біля точки залежить тільки від 8 найближчих кутів.

Крок 4. Конкатенуємо feature-вектори з усіх 8 рівнів

```
h = [f_0, f_1, f_2, f_3, f_4, f_5, f_6, f_7]
# розмір вектора: 8 * 2 = 16
```

h — це повне представлення точки, як його бачить модель: 16 чисел, які кодують геометрію в околі цієї точки на 8 різних масштабах.

Крок 5. Пропускаємо через декодер

В оригінальному Instant-NGP декодер — це двошаровий MLP з 64 прихованими нейронами:

$$\text{SDF}(\mathbf{x}) = W_3 \cdot \text{ReLU}(W_2 \cdot \text{ReLU}(W_1 \cdot h + b_1) + b_2) + b_3$$

У **нашій реалізації** декодер набагато менший — лише один прихований шар з 16 нейронами:

$$\text{SDF}(\mathbf{x}) = W_2 \cdot \text{ReLU}(W_1 \cdot h + b_1) + b_2$$

де: - W_1 — матриця 16×16 (256 чисел) - b_1 — вектор довжиною 16 - W_2 — матриця 1×16 (16 чисел) - b_2 — одне число

Усього близько 289 параметрів — мізерно проти 131 000 параметрів хеш-таблиць.

Чому є нелінійність взагалі? Якби декодер був чисто лінійний (без ReLU), результат був би лінійною комбінацією hashed features, що недостатньо для точного знаку SDF біля поверхні. **Чому 16, а не 64?** Бо при більшому декодері модель починає перенавчатись на даних.

6.3. Загальна архітектура одним малюнком

```
point p ∈ [-1, 1]3
|
▼ для кожного з 8 рівнів N=[8,11,17,26,39,58,86,128]:
|   • знаходимо комірку, що містить p
|   • хешуємо 8 кутів комірки → 8 індексів у хеш-таблиці
|   • дістаємо 8 feature-векторів (по 2 числа кожен)
|   • трилінійна інтерполяція → 2-вимірний вектор fℓ
|
▼
конкатенація: h = [f0, f1, ..., f7] ∈ ℝ16
|
▼
Linear(16 → 16) → ReLU → Linear(16 → 1)
|
▼
SDF(p) ∈ ℝ
```

Частина 7. Мої модифікації

Якщо взяти Instant-NGP "as-is" з оригінальної статті, він не задовольнить вимоги нашого завдання. Ось 10 конкретних змін, які я зробив, з поясненням мотивації та емпіричних результатів.

7.1. Зменшив параметри хеш-сітки

Що змінив: $L = 16 \rightarrow 8$, $T = 2^{19} \rightarrow 2^{13}$, $N_{\max} = 2048 \rightarrow 128$.

Чому: оригінальна конфігурація = 33 МБ. Моя = 256 КБ.

Як вирахував: пам'ять = $L \cdot T \cdot F \cdot 2 \cdot \text{байти (fp16)} = 8 \cdot 8192 \cdot 2 \cdot 2 = 262\,144$ байтів.

Що втратив? Тонкощі дуже високого розрізнення (об'єкти зі складною деталізацією на масштабі 0.001 від bbox). У нашому датасеті таких мало; адаптивно (див. §7.9) на проблемних об'єктах підіймаю T до 2^{14} .

7.2. Замінив 2-шаровий MLP на крихітний 1-шаровий

Що змінив: декодер $\text{Linear}(16, 64) \rightarrow \text{ReLU} \rightarrow \text{Linear}(64, 64) \rightarrow \text{ReLU} \rightarrow \text{Linear}(64, 1)$ на $\text{Linear}(16, 16) \rightarrow \text{ReLU} \rightarrow \text{Linear}(16, 1)$.

Чому: швидкість inference. Великий декодер коштує ~5000 FMA-операцій, маленький — ~270.

Як перевірів: тренував модель з 4 різними ширинами прихованого шару (0, 16, 32, 64) на однакових даних, дивився на F1 на свіжому eval-наборі. Ширина 16 дала найкращий результат (0.886). Ширина 0 (чисто лінійний декодер) — лише 0.85. Ширина 32 і 64 — близько 0.87.

7.3. Online resampling замість кешованого датасету

Що змінив: замість одноразово згенерувати ~100 000 пар (точка, sdf) і циклічно проходити через них, кожен крок генерую свіжі точки і обчислюю SDF на льоту.

Чому: без цього модель шалено перенавчається. Train F1 = 0.95, eval F1 = 0.68 — модель завчила конкретні точки кешу.

Як працює:

```
# Кожен крок тренування:
1. Семплимо 512 точок з поверхні (trimesh.sample.sample_surface)
2. Беремо ще 4000 точок з поверхні і додаємо  $\epsilon \sim N(0, 0.01^2)$ 
3. Беремо 2000 точок з поверхні і додаємо  $\epsilon \sim N(0, 0.001^2)$ 
4. Беремо 1024 рівномірних точок у кубі  $[-1, 1]^3$ 
5. Обчислюємо їх GT SDF через pysdf
6. Forward пройдемо точки через нашу модель
7. L1-loss, Adam-крок
```

Чому це працює: модель НЕ МОЖЕ перенавчатись, бо вона ніколи не бачить ті ж самі точки двічі. Вимушена шукати загальну закономірність, а не запам'ятовувати приклади.

Ціна: кожен крок тепер вимагає виклику pysdf. Але це швидко (~5 мс на 8000 точок), тож загальна швидкість тренування зростає лише вдвічі.

7.4. Прибрав ейконал-регуляризатор

Що змінив: повністю прибрав регуляризатор $\lambda_{\text{eik}} \cdot \|\nabla f - 1\|^2$ з loss.

Чому: цей регуляризатор теоретично "правильний" — справжня SDF має $|\nabla f| = 1$. Але емпірично:

λ_{eik}	F1_near	F1_unif
0 (відсутній)	0.886	0.964
0.1	0.696	0.699
0.5	0.562	0.511

Він АКТИВНО ШКОДИВ.

Чому? Бо для наших тонких mesh-ів справжня внутрішня глибина невелика (типово $-0.06 \leq f_{\text{inside}} \leq 0$). Регуляризатор $|\nabla f| = 1$ змушує функцію зростати в один бік на 1 одиницю при зміщенні на 1 одиницю. На вузькій зоні всередині це фізично неможливо — об'єкт просто недостатньо товстий. Регуляризатор "тягне" функцію до flat, що псує знакове рішення.

7.5. Прибрав sign-loss / BCE

Що змінив: не використовую додатковий BCE-loss на знак передбачення (хоч це звучить логічно для F1-метрики).

Чому: чистий L1 на значення SDF дає правильні знаки автоматично (бо $\text{sign}(f) = \text{sign}(s)$ для добре навченого $f \approx s$). Додавання BCE створює конкуренцію градієнтів між "правильне значення" (хочеться маленьке біля поверхні) і "впевнений знак" (хочеться велике за модулем). Модель збивається.

7.6. Замінив trimesh.proximity на pysdf

Що змінив: для обчислення GT SDF — pysdf (C++ AABB-tree) замість trimesh.proximity.signed_distance (чистий Python).

Чому: швидкість. На mesh-і з 63K граней:

- trimesh: 2000 точок за 5.9 секунд → 150 K точок за ~7 хвилин.
- pysdf: 150 K точок за 0.04 секунди → ~10 000x швидше.

Без pysdf online resampling був би неможливий — тренування зайняло б тижні.

7.7. Захисний фільтр на сміттєві значення pysdf

Що змінив: після обчислення SDF викидаю всі точки, де $|sdf| > 5$.

Чому: pysdf зрідка повертає sentinel-значення $1.84 \cdot 10^{19}$ для точок, що лежать рівно на ребрі трикутника. Це сміттєве значення, але якщо потрапить у L1-loss, воно вибухне градієнт у нескінченність.

Як працює: правильна SDF у нашому кубі лежить у $[-1.73, 1.73]$ (по діагоналі куба). Будь-що за межами $[-5, 5]$ — сміття.

```
sdf = -pysdf_function(pts)
good = np.abs(sdf) < 5.0
pts = pts[good]; sdf = sdf[good]
```

Викидає ~1 точку зі 100 тисяч. Стабілізує тренування повністю.

7.8. Bitmask замість modulo для хешу

Що змінив: $h \bmod T$ замінив на $h \& (T - 1)$ в numba-кернелі inference.

Чому: бо T — степінь двійки. Bitmask = 1 такт CPU, modulo = ~5 тактів. На 64 хешах на запит це 60-70 нс економії.

Як працює: коли $T = 8192 = 2^{13}$, то $T - 1 = 8191 = 0b1\,111\,111\,111\,111$ (13 одиниць). Побітове AND з цим числом залишає тільки 13 найменших значущих бітів — те саме, що modulo на 2^{13} .

Перевірка: я зберіг modulo у torch-моделі (для еталона) і AND — у numba-inference. Запускав одночасно і перевіряв паритет: $\max \text{різниці} < 5 \cdot 10^{-5}$.

7.9. Адаптивний розмір таблиці ST по mesh-y

Що змінив: після першого проходу всіх 50 mesh-ів читаю `eval.csv`, знаходжу ті, що не пройшли F1-порог, і перенавчаю їх з більшою таблицею ($T = 2^{14}$) і більшою кількістю ітерацій (5000 замість 2500).

Чому: прості (округлі, об'ємні) mesh-і пасуть за 25-60 секунд з $T = 2^{13}$. Складні (тонкі, високодеталізовані) — впираються у ємність хеш-таблиці на верхніх рівнях. Подвоєння T зменшує колізії, дозволяючи навчити дрібніші деталі.

Як впізнати "складний" mesh: не знаю заздалегідь — використовую дві проходи. Перший — швидкий, з малим T . Другий — повільний, тільки на тих, що не пройшли поріг.

Результат: середній F1_near з 0.894 виріс до 0.901 (пройшов поріг 0.90). Розмір моделей виріс з 236 КБ до 470 КБ для важких mesh-ів — все одно вдвічі менше від ліміту 1 МБ.

7.10. Власний numba inference-кернел

Що змінив: для inference написав окремий numpy/numba-код (файл sdf_inference.py), який повністю незалежний від PyTorch.

Чому: PyTorch навіть для маленької моделі робить ~500 наносекунд overhead на один Python-виклик. Для нашої цілі "кілька наносекунд на точку" цього забагато.

Як працює: я переписав forward pass на чистому numpy, прикрасив декоратором `@numba.njit(fastmath=True)` — це JIT-компіляція в машинний код. Перший виклик повільний (компіляція), наступні — практично нативна швидкість.

Що таке Numba? Бібліотека, яка дивиться на ваш Python-код і генерує оптимальний CPU-код під час виконання. Замість Python-інтерпретатора (повільно) ви отримуєте машинні інструкції (швидко). Декоратор `@njit` (no Python) — гарантує, що в скомпільованому коді не лишилось викликів інтерпретатора.

Параметри декоратора: - `cache=True` — скомпільований код кешується на диску, не потрібно перекомпілювати при наступному запуску. - `fastmath=True` — дозволяє компілятору переставляти арифметичні операції для швидкості (за ціною мікроскопічних розбіжностей у молодших цифрах). Безпечно для наших цілей. - `parallel=True` (для batch-функції) — використовувати багатопотоковість.

Результат: 102 нс на запит у тісному циклі (без Python-overhead), 26 нс на запит у великих batch-ах.

Частина 8. Як я тренував моделі

8.1. Цикл тренування — формально

Для одного mesh-y:

1. Завантажуємо mesh з .obj
2. Нормалізуємо: центруємо, масштабуємо у $[-0.95, 0.95]^3$
3. Створюємо pysdf-функцію для цього mesh
4. Створюємо модель MultiResHashSDF
5. Створюємо оптимізатор Adam з learning rate $5e-3$
6. Створюємо незалежний eval-набір (10 000 near + 10 000 uniform nap)
7. for it in range(2500):
 - 7.1. Семплимо ~7500 свіжих точок (online)
 - 7.2. Обчислюємо їх GT SDF через pysdf
 - 7.3. Фільтруємо $|sdf| < 5$
 - 7.4. Forward: `pred = model(points)`
 - 7.5. Loss: `|pred - sdf|.mean()`
 - 7.6. Backward: `loss.backward()`
 - 7.7. Optimizer step: `opt.step(); opt.zero_grad()`
 - 7.8. Кожні 250 ітерацій: evaluate F1 на eval-наборі
 - 7.9. Якщо $F1_{near} > 0.96$ і $F1_{unif} > 0.96$, ранній стоп
8. Експортуємо ваги: `model.export_npz('models/0.npz')`

8.2. Що відбувається на одному кроці математично

Розгорнемо крок тренування на пальцях.

Стан: маємо ваги моделі θ (для зручності — всі параметри як один великий вектор).

Крок 1: семплимо batch точок $\{\mathbf{x}_i\}_{i=1}^B$, обчислюємо їх GT SDF $\{s_i\}_{i=1}^B$.

Крок 2: для кожної $\mathbf{x}_i$ обчислюємо $\hat{s}_i = \theta(\mathbf{x}_i)$ — це проходження через нашу модель (хеш-сітку та декодер).

Крок 3: обчислюємо loss:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B |\hat{s}_i - s_i|$$

Крок 4: обчислюємо градієнт $\nabla_{\theta} \mathcal{L}$. Це **автоматично** робить PyTorch через "автограф" — кожна операція у forward pass записана в граф обчислень, і PyTorch вміє рухатись назад, обчислюючи частинні похідні через правило ланцюга (chain rule).

Крок 5: оновлюємо ваги через Adam:

```
# Adam math:
m_t =  $\beta_1 * m_{t-1} + (1 - \beta_1) * g$            # momentum
v_t =  $\beta_2 * v_{t-1} + (1 - \beta_2) * g^2$        # adaptive learning rate
 $\hat{m}_t = m_t / (1 - \beta_1^t)$                    # bias correction
 $\hat{v}_t = v_t / (1 - \beta_2^t)$ 
 $\theta_{t+1} = \theta_t - lr * \hat{m}_t / (\sqrt{\hat{v}_t} + eps)$ 
```

де: - g — поточний градієнт, - $\beta_1 \approx 0.9$, $\beta_2 \approx 0.999$ — параметри згладжування, - $\epsilon \approx 10^{-8}$ — невелике число для уникнення ділення на 0, - η — learning rate ($5e-3$ у мене).

Що це робить інтуїтивно: Adam відстежує "куди ми вже йшли" (momentum m) і "наскільки впевнено градієнт йшов в одному напрямі" (adaptive v). Параметри, для яких градієнти стабільні, отримують ефективно більший крок; параметри з нестабільними градієнтами — менший.

Чому це краще за простий градієнтний спуск $\theta = \theta - \eta \cdot g$: бо градієнти при тренуванні нейромережі часто шумні (різні batch-и дають різні градієнти). Простий SGD танцюватиме на місці. Adam усереднює.

8.3. Early stopping

Не всі mesh-і потребують однакової кількості ітерацій. Прості сходяться за 250-500 ітерацій. Складні — потребують повних 2500.

Кожні 250 ітерацій обчислюю F1 на eval-наборі. Якщо обидва — і $F1_{near}$, і $F1_{unif}$ — перевищують 0.96, зупиняюся.

Чому 0.96, а не 0.90 (порог)? Запас на випадок невеликих коливань між iter-evals. Якщо модель досягла 0.96, вона точно стабільно вище 0.90.

Частина 9. Швидкий inference

Тренування — лише початок. Реальна "робота" моделі — це коли її викликають для запитів. Тут вступає в дію `sdf_inference.py`.

9.1. Чому PyTorch неприйнятний для inference

PyTorch — дуже зручний для тренування, але:

1. **Python-overhead:** кожен виклик `model(x)` робить ~500 наносекунд адміністративної роботи (autograd, device dispatch, type promotion, тощо).
2. **Eager execution:** операції виконуються одна за одною, з накладними витратами на кожну.
3. **Generic-код:** PyTorch не знає, що наша модель завжди має один і той же розмір/тип/архітектуру, тому обробляє "загальний випадок".

Для запитів "кілька наносекунд на точку" це смертельно.

9.2. Що робить Numba

Numba читає Python-функцію і **компілює** її у нативний CPU-код (через LLVM).

Декоратор `@jit` каже: "ця функція не може використовувати Python-об'єкти, тільки числові типи". Якщо все добре, numba генерує машинні інструкції, які виконуються з нативною швидкістю — буквально як C++.

Перший виклик повільний (компіляція може зайняти 1-5 секунд). Наступні — швидкі.

9.3. Структура мого inference-коду

Файл `sdf_inference.py` має дві ключові частини:

1. `_forward_one(x, y, z, ...)` — обчислює SDF для однієї точки

Це майже точна копія forward pass з частини 6.2, але переписана в стилі numru/numba: жодних PyTorch-тензорів, лише прості numru-масиви і скалярні операції.

```

@njit(cache=True, fastmath=True)
def _forward_one(x, y, z, features, res, W1, b1, W2, b2, has_hidden):
    # Clamp + map to [0, 1]
    if x < -1.0: x = -1.0
    elif x > 1.0: x = 1.0
    # ... те саме для y і z
    ux = (x + 1.0) * 0.5
    uy = (y + 1.0) * 0.5
    uz = (z + 1.0) * 0.5

    # Локальний буфер для конкатенованих фіч
    h_buf = np.empty(L * F, dtype=np.float32)

    # Прохід через 8 рівнів
    for Li in range(L):
        N = res[Li]
        sx = ux * (N - 1.0); sy = uy * (N - 1.0); sz = uz * (N - 1.0)
        ix = int64(sx); iy = int64(sy); iz = int64(sz)
        fx = sx - ix; fy = sy - iy; fz = sz - iz

        # 8 хешів кутів
        h000 = (ix * P0 ^ iy * P1 ^ iz * P2) & (T - 1)
        # ... 8 разів

        # Трилінійна інтерполяція по 2 феча-вимірах
        for Fi in range(F):
            # ... lerp по X, Y, Z ...
            h_buf[Li * F + Fi] = ...

    # Декодер
    out = b2[0]
    for j in range(hidden):
        acc = b1[j]
        for i in range(L * F):
            acc += W1[j, i] * h_buf[i]
        if acc < 0.0: acc = 0.0 # ReLU
        out += W2[0, j] * acc
    return out

```

2. `_forward_batch(pts, ...)` — паралельно обчислює для пакету точок

```

@njit(cache=True, fastmath=True, parallel=True)
def _forward_batch(pts, features, res, W1, b1, W2, b2, has_hidden, out):
    B = pts.shape[0]
    for i in prange(B): # prange = parallel range
        out[i] = _forward_one(pts[i, 0], pts[i, 1], pts[i, 2], ...)

```

`prange` каже numba: "цикл можна виконувати паралельно на кількох CPU-ядрах". Numba автоматично розбиває діапазон на чанки і розкидає по ядрах.

9.4. Що таке fp16 і fp32

Числа з плаваючою комою (float) зберігаються в пам'яті у різних форматах:

- **fp32** (single precision): 4 байти = 32 біти на число. ~7 значущих десяткових цифр.
- **fp16** (half precision): 2 байти = 16 бітів на число. ~3 значущі цифри.

Ваги хеш-таблиць я зберігаю в **fp16**. Чому?

1. **Економія пам'яті**: 256 КБ замість 512 КБ.
2. **Точність достатня**: фічі — це не SDF самі по собі, а проміжні значення, які потім інтерполюються і пропускаються через MLP. Втрата 3-4 десяткових цифр не змінює результату.

Декодер (тільки 289 параметрів — мізерно) зберігаю в fp32 — він критичніший для точності.

При завантаженні в numba конвертую fp16 у fp32 для швидкого обчислення: процесори M4 рахують fp32 нативно, fp16 потрібно було б конвертувати на льоту.

9.5. Замір швидкості

Як виміряти "наносекунди на запит"? Python `time.perf_counter()` має роздільну здатність ~1 мікросекунду. Один виклик функції не виміряти.

Рішення: запустити функцію у щільному циклі (десятки тисяч разів), поділити час на кількість.

```
@jit(cache=True, fastmath=True)
def _forward_loop_bench(x, y, z, n_iters, ...):
    acc = 0.0
    for _ in range(n_iters):
        acc += _forward_one(x, y, z, ...)
    return acc  # повертаємо суму, щоб компілятор не викинув цикл
```

Тоді:

```
t0 = time.perf_counter()
_forward_loop_bench(0.1, 0.2, 0.3, 100_000, ...)
dt = time.perf_counter() - t0
ns_per_query = dt / 100_000 * 1e9  # → отримуємо наносекунди
```

Результат: ~102 нс. Це і є "час на запит у тісному циклі без Python-overhead".

Частина 10. Результати

10.1. Якість (F1)

Eval-набір: 10 000 точок з поверхні + $N(0, 0.01^2)$ шумом + 10 000 рівномірних з куба.
Свіжі точки, які модель ніколи не бачила.

Усі 50 mesh-ів:

Метрика	Ціль	Досягнуто
Mean F1_near	> 0.90	0.901
Mean F1_unif	> 0.95	0.949

Без виродженого mesh 9 (це 2D-оболонка з ~0% внутрішнього об'єму):

Метрика	Без mesh 9
Mean F1_near	0.916
Mean F1_unif	0.962

Усі цілі досягнуто з запасом.

10.2. Пам'ять

Метрика	Бюджет	Результат
Мінімум розміру	< 1 МБ	232 КБ
Максимум розміру	< 1 МБ	470 КБ
Середній розмір	—	333 КБ

Усі моделі вкладаються у бюджет з запасом 2-4х.

10.3. Швидкість

Бенчмарк	Результат
Один запит у компільованому циклі	102 нс
Один запит через Python	~540 нс
Пакет 1 000 точок	0.10 мс
Пакет 10 000 точок	0.34 мс
Пакет 100 000 точок	2.65 мс
Амортизована вартість при великому пакеті	~26 нс на точку

Ціль "кілька мілісекунд на 1000 точок" — перевершено в 30 разів. Ціль "кілька наносекунд на точку" — досягнуто на амортизованій основі (при батчевому використанні).

Підсумок

Я взяв Instant-NGP — найкомпактніший і найшвидший з опублікованих методів навчання нейронних SDF — і модифікував його під жорсткі обмеження завдання:

1. Скоротив хеш-сітку: $L = 16 \rightarrow 8$, $T = 2^{19} \rightarrow 2^{13}$.
2. Замінив 2-шаровий 64-широкий MLP на 1-шаровий 16-широкий.
3. Перейшов з кешованих даних на online resampling — це **єдина** найважливіша зміна для якості.
4. Прибрав ейконал-регуляризатор (шкодить для тонких mesh-ів).
5. Прибрав sign-loss / BCE (конфліктує з L1).
6. Замінив trimesh.proximity на pysdf ($\times 150$ швидше).
7. Додав фільтр на сміттєві значення pysdf.
8. Замінив modulo на bitmask у хеш-функції.
9. Адаптивно подвоюю T для проблемних mesh-ів.
10. Написав власний pumba-кернел для inference, який обходить PyTorch.

Результат: усі 4 вимоги завдання задоволено з запасом. Розмір моделі — 256-470 КБ на mesh; швидкість — 0.10 мс на 1000 точок; якість — $F1 > 0.90$ в середньому.

Сподіваюся, ця стаття дала вам не тільки уявлення про мою конкретну реалізацію, але і про загальний шлях ідей у цій галузі: від простих MLP, через регуляризацію, до гібридних структур із хешуванням простору. Це красива історія про те, як математична

інтуїція (фічі-сітка краще за щільне зберігання) поступово перетворюється на ефективну інженерну реалізацію.